

グラフ理論 木オイラーツアー LCA（最小共通祖先）スパーステーブル

オイラーツアーテクニック

[解説動画はこちら](#)です。

1. 概要

本講座では、**オイラーツアーテクニック** について解説します。

オイラーツアーテクニックは [論文 3](#) において導入されたテクニックです。これは木の情報を、頂点全体や辺全体を特定の方法で 1 列に並べることによって表現するものです。このテクニックによって、木に対するさまざまな問題を列に対する問題へと変換できます。

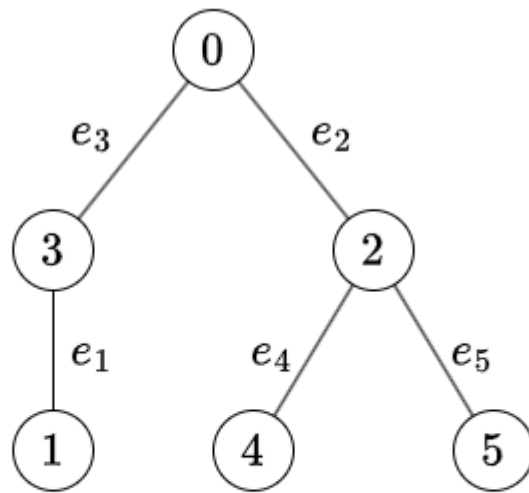
本講座では特に、部分木クエリ、パスクエリ、最小共通祖先クエリへの応用を扱います。

2. 前提知識

DFS. セグメント木などのデータ構造。

3. オイラーツアーテクニック

根付き木に対して、根から DFS を行うことで、全ての辺を双方向に 1 度ずつ通るようなウォークが得られます。言い換えればこのウォークは、木の各辺を双方向の有向辺に置き換えたときのオイラーウォークです。このウォークを**オイラーツアー**（Euler Tour）と呼びます。



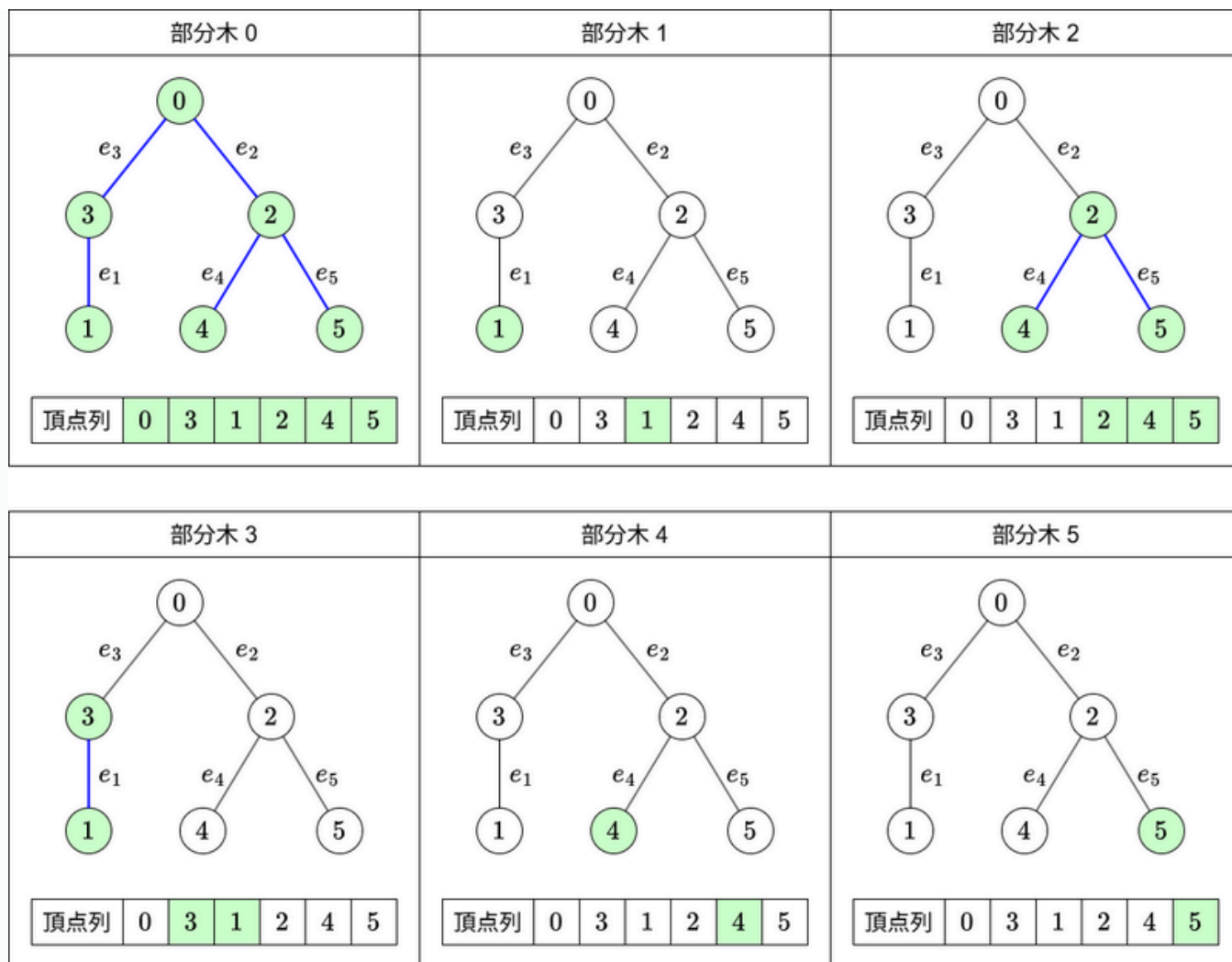
頂点列	0	3	1	3	0	2	4	2	5	2	0
辺列	e_3	e_1	e_1	e_3	e_2	e_4	e_4	e_5	e_5	e_2	

この頂点列や辺列，あるいはその一部を利用するテクニックを，**オイラーツアーテクニック**（Euler Tour Technique）といいます。

4. 部分木クエリ

オイラーツアーでは，同じ辺を双方向に 1 度ずつ通ります。したがってある頂点 v の部分木に入ると，その部分木全体をってから，頂点 v の部分木を出るという動きになります。特に，オイラーツアーにおいて部分木 v 内にある部分は区間になります。このことから，部分木に関するクエリを区間に関するクエリに変換できます。

この際，オイラーツアーに同じ頂点が複数回現れるという冗長さを除くために，各頂点の最初の出現のみを並べた頂点列（**行きがけ順**，**pre-order**）がよく用いられます。



各頂点 v に対して、部分木 v と対応する区間インデックスを求めるには、次の擬似コードで示すような実装をすればよいです。

```

in := array of length N
out := array of length N
tour := empty list

procedure dfs(v):
  in[v] := length(tour)
  append v to tour
  for each child w of v:
    dfs(w)
  out[v] := length(tour)

```

v への初訪問時に tour に v を追加しています。 $\text{in}[v]$, $\text{out}[v]$ には部分木 v の処理が始まる時点、終了する時点での tour の長さが入ります。したがって、部分木 v には半开区間 $[\text{in}[v], \text{out}[v])$ が対応します。

このことを利用すると、例えば次のような問題を解くことができます。

【問題 1】

N 頂点の根付き木が与えられます。頂点 v は重み a_v を持ちます。次の形式の Q クエリを処理してください。

- 0 $v\ x$: a_v に x を加える。
- 1 v : v の部分木に含まれる頂点の重みの総和を出力する。

セグメント木や Fenwick 木を用いると、 $O(N + Q \log N)$ 時間でこの問題を解くことができます。

4.1. 実装例

Library Checker [Vertex Add Subtree Sum](#) への次の提出を参考にしてください。

- <https://judge.yosupo.jp/submission/378382>

5. パスクエリ

今度は、辺の列を使ってみましょう。

辺の列に関するオイラーツアーは、次の擬似コードのように実装することができます。このまま実装すると、「根とその親を結ぶ辺」も列に追加されるので、必要があれば v が根の場合の追加を省略してください。

```
in := array of length N
out := array of length N
tour := empty list

procedure dfs(v):
    append edge (parent[v] to v) to tour
    in[v] := length(tour)
    for each child w of v:
        dfs(w)
```

```
out[v] := length(tour)
append edge (v to parent[v]) to tour
```

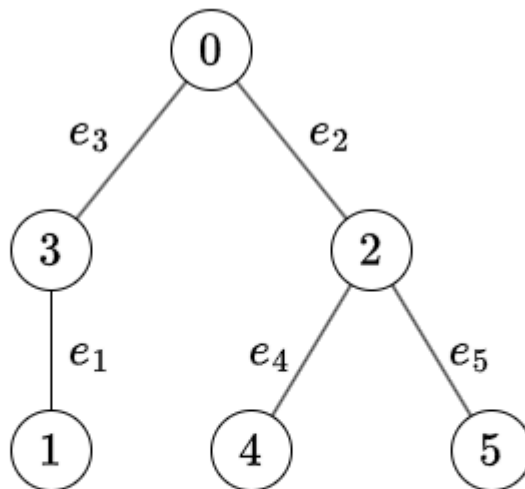
次のような問題を考えます. 子側の頂点が v であるような辺を e_v と書くことにします.

【問題 2】

N 頂点の根付き木が与えられます. 辺 e_v は重み a_v を持ちます. 次の形式の Q クエリを処理してください.

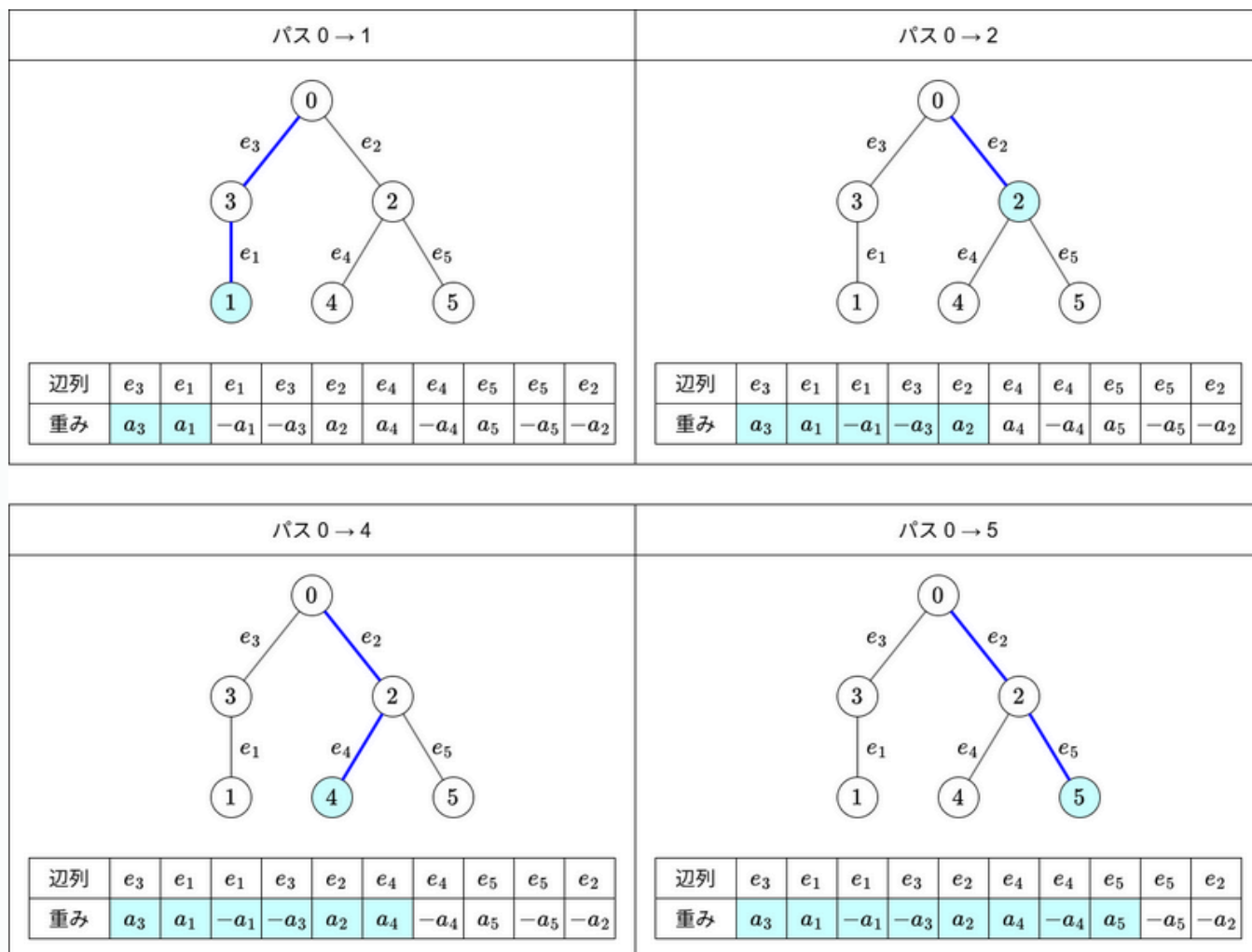
- 1 $v\ x: a_v$ を x に変更する. ただし v は根ではない.
- 2 v : 根から v までのパスが通る辺の重みの総和を出力する.

オイラーツアーにおける辺の列をとり, さらに親から子への移動, 子から親への移動を区別して扱います. v の親から v へ移動する辺に対して重み $+a_v$, v から v の親へ移動する辺に対して重み $-a_v$ を持たせてみましょう.



辺列	e_3	e_1	e_1	e_3	e_2	e_4	e_4	e_5	e_5	e_2
重み	a_3	a_1	$-a_1$	$-a_3$	a_2	a_4	$-a_4$	a_5	$-a_5$	$-a_2$

すると, 根から v までのパスが通る辺の重みの総和は, オイラーツアーにおいて v に到達するまでに通る辺の重みの総和に一致します. 双方向に通る辺の重みが打ち消しあって, 根から v までの辺の重みのみが残るからです.



このことを利用すると，【問題 2】 はセグメント木や Fenwick 木を利用して $O(N + Q \log N)$ 時間で解くことができます。

▶ 注意 1

▶ 注意 2

▶ 注意 3

▶ 注意 4

6. LCA クエリ

【問題 3】

N 頂点の根付き木が与えられます. 次の形式の Q クエリを処理してください.

- $u\ v$: u, v の LCA を出力する.

u, v を 2 頂点とし, オイラーツアーにおけるそれぞれの出現位置のひとつ (例えば最初の出現位置) に注目します. オイラーツアーにおいて u が v より先に現れるとし, u の出現位置を l , v の出現位置を r とします. w を u, v の LCA とするとき, 次のことが成り立ちます:

- オイラーツアーにおける l 番目から r 番目までのウォークの部分は, w を通る.
- オイラーツアーにおける l 番目から r 番目までのウォークの部分は, 部分木 w に含まれる.

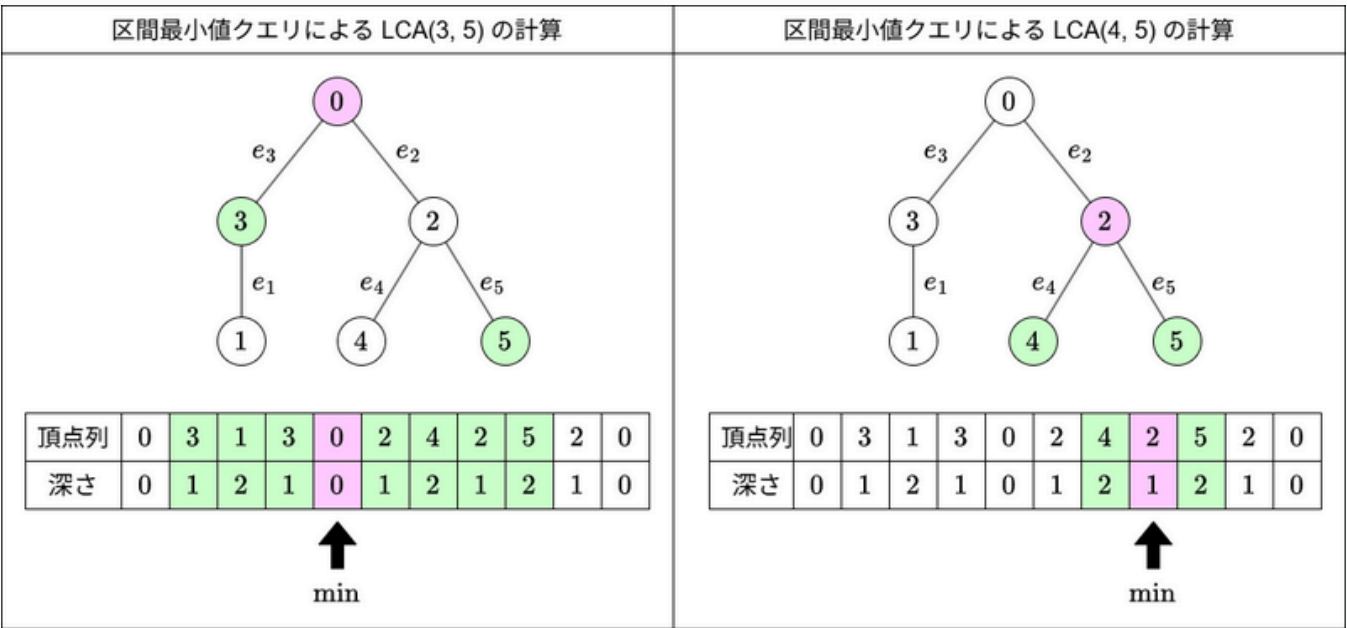
前者はパス $u \rightarrow v$ が w を通るからです. 後者はオイラーツアーのうち部分木 w に含まれる部分が区間をなすことから分かります.

したがって, 次のことが成り立ちます: u, v の最小共通祖先は, オイラーツアーにおける u から v までのウォークの部分における, 深さ最小の頂点である.

よって, LCA クエリに答えるには,

- オイラーツアーの頂点列を (2 度目以降の出現を省略することなく) 列挙する.
- 対応する深さの列に対する, 区間最小値クエリを処理する.

とすればよいです.



オイラーツアーの頂点列を列挙するには、次の擬似コードのような実装をすればよいです。

```
tour := empty list

procedure dfs(v):
    append v to tour
    for each child w of v:
        dfs(w)
    append v to tour
```

以上により、【問題 3】は

- セグメント木を用いると、 $O(N + Q \log N)$ 時間.
- スパーステーブルを用いると、 $O(N \log N + Q)$ 時間.

で解くことができます.

7. 参考：オイラーツアー木

木をオイラーツアー、特に有向辺の列として表し、この列を平衡二分木で管理することで、**動的木**のデータ構造が得られます。動的木というのは、木に対する辺の削除・追加が行えるということです。このデータ構造は**オイラーツアー木**（Euler Tour Tree）と呼ばれます。

木のオイラーツアーを指してオイラーツアー木と呼ぶのは誤用なので注意してください。オイラーツアー木については、AtCoder Algorithm Lectures でも今後解説する予定です。

8. 関連問題

1. https://judge.yosupo.jp/problem/vertex_add_subtree_sum
2. <https://cses.fi/problemset/task/1137>
3. <https://cses.fi/problemset/task/1138>
4. <https://judge.yosupo.jp/problem/lca>
5. https://atcoder.jp/contests/abc294/tasks/abc294_g

▶ 問題について

9. まとめ

本講座では、頂点全体や辺全体を特定の方法で 1 列に並べることによって木の情報を表す、オイラーツアーテクニックについて、部分木クエリ、パスクエリ、LCA クエリへの応用を解説しました。

特に、部分木クエリは競技プログラミングでも頻出です。パスクエリについては 重軽分解 (HLD) を利用する方法が強力なので、競技プログラミングで必須となることは多くありません。

LCA クエリについて、本講座の手法は、【問題 3】 を $O(N + Q)$ 時間で解く 線形時間 LCA の解法に応用されます。

10. 参考文献

1. Wikipedia. Euler tour technique.
https://en.wikipedia.org/wiki/Euler_tour_technique. (閲覧日: 2026-03-31).
2. USACO Guide. Euler Tour Technique. <https://usaco.guide/gold/tree-euler>. (閲覧日: 2026-03-31).
3. Robert E. Tarjan, Uzi Vishkin. Finding Biconnected Components and Computing Tree Functions in Logarithmic Parallel Time (Extended Summary). Proceedings of 25th Annual Symposium on Foundations of Computer Science (FOCS). pp. 12–20. 1984. DOI: 10.1109/SFCS.1984.715896.
<https://doi.org/10.1109/SFCS.1984.715896>.
4. maspy. Euler Tour のお勉強. <https://maspy.py.com/euler-tour-のお勉強>. (閲覧日: 2026-03-31).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)